# AI-Powered Software Engineering Study Assistant

**BAWAZIR AHMED ALI AHMED**

**November 2025**

# AI-Powered Software Engineering Study Assistant

**BAWAZIR AHMED ALI AHMED**

**A Project Report Submitted to the**
**College of Computing and Informatics**
**Universiti Tenaga Nasional**
**in Partial Fulfilment of the Requirements for the**
**Bachelor's in Computer Science (Software Engineering) (Hons.)**

**NOVEMBER 2025**

# DECLARATION

I hereby declare that final year project is my original work except for quotations and citations have been duly acknowledged. I also declare that has not been previously and is not concurrently submitted for any degree program at Universiti Tenaga Nasional or at any other institutions. This final year project may be made available within the university library and may be borrowed, consulted, copied, or reproduced in accordance with the provision of the UNITEN Library Regulations from time to time made by the Library Committee.

BAWAZIR AHMED ALI AHMED

SW01084040

30/10/2025

APPROVAL PAGE

**This thesis entitled:**

**"AI-Powered Software Engineering Study Assistant"**

**submitted by:**

**BAWAZIR AHMED ALI AHMED (SW01084040)**
**In requirement for the degree of Bachelor in Computer Science (Software Engineering) (Hons.) College of Computing and Informatics, Universiti Tenaga Nasional has been accepted.**

**Supervisor: Prof. Dr. Ahmed Mubarak Al-Haiqi**
**Signature: …….**
**Date: 30/10/2025**

# ACKNOWLEDGMENT

First and foremost, I thank Allah Almighty for giving me the strength, patience, and guidance to complete this project. Without His blessings, this journey would not have been possible.

I would like to express my sincere appreciation to my supervisor, **Prof. Dr. Ahmed Mubarak Al-Haiqi**, for his continuous support, advice, and encouragement throughout the development of this project. His guidance helped me stay focused and allowed me to improve both the technical and academic quality of my work.

My gratitude also goes to the lecturers and staff of the **College of Computing and Informatics, Universiti Tenaga Nasional (UNITEN)** for providing the knowledge, facilities, and learning environment that supported my growth as a student.

I am especially grateful to my classmates and friends who encouraged me, shared ideas, participated in the survey and interviews, and supported me whenever I needed help. Their feedback played an important role in shaping the AI-Powered Software Engineering Study Assistant into a useful and meaningful system.

Lastly, I want to thank my family for their constant support, understanding, and motivation throughout my academic journey. Their belief in me has always been a source of strength, and I am truly grateful for their endless encouragement.

# ABSTRACT

This project presents the design and development of an AI-Powered Software Engineering Study Assistant aimed at supporting undergraduate students throughout the Software Development Life Cycle (SDLC). Many students struggle with key SDLC tasks such as generating project ideas, writing clear requirements, constructing UML diagrams, designing database schemas, reviewing code, and creating structured test cases. To address these challenges, this project integrates a Large Language Model (LLM) to provide real-time, context-aware guidance and automated generation of essential software engineering artefacts. The methodology involved requirements elicitation through questionnaires and interviews with software engineering students, followed by system analysis, architectural design, interface design, and database modelling. The assistant was developed as a web-based platform incorporating modules for idea generation, requirements analysis, UML generation, database schema recommendation, code generation, code review, and test case generation. Each module leverages natural language processing to transform user input into structured outputs aligned with SDLC best practices. The system additionally supports file export to assist students with documentation and project reporting. The results of the project demonstrate that an AI-driven assistant can simplify complex SDLC tasks, reduce student workload, and improve understanding of software development concepts by providing immediate feedback and structured artefacts. Although full deployment and user testing will be completed in the second phase of the Final Year Project, the current prototype establishes a solid foundation for an intelligent educational tool capable of enhancing learning outcomes. This project contributes to improving the accessibility of software engineering practices for students and highlights the potential of AI-assisted learning environments in academic settings.

## TABLE OF CONTENTS


| | | Page |
|---|---|---|
| **DECLARATION** | | **ii** |
| **APPROVAL PAGE** | | **iii** |
| **ACKNOWLEDGMENT** | | **iv** |
| **ABSTRACT** | | **v** |
| **TABLE OF CONTENTS** | | **vi** |
| **LIST OF FIGURES** | | **x** |
| **LIST OF SYMBOLS** | | **xii** |
| **LIST OF ABBREVIATIONS** | | **xiii** |
| **CHAPTER 1** | **INTRODUCTION** | **1** |
| 1.1 | Project Background | 1 |
| 1.2 | Problem Statement | 2 |
| 1.3 | Objective | 2 |
| 1.4 | Scope | 3 |
| **CHAPTER 2** | **LITERATURE REVIEW** | **5** |
| 2.1 | Related Works | 5 |
| 2.1.1 | GitHub Copilot | 5 |
| 2.1.2 | ChatGPT | 6 |
| 2.1.3 | Ask SIA (Student Intelligent Assistant) | 7 |
| 2.1.4 | Visual Paradigm | 7 |
| 2.1.5 | BlueJ | 8 |
| 2.2 | Findings and Analysis | 8 |
| 2.3 | Summary of Findings | 9 |
| 2.4 | Gap in Existing Systems | 9 |
| **CHAPTER 3** | **REQUIREMENT ANALYSIS** | **11** |

3.1 Requirements Elicitation 11

3.1.1 Elicitation Technique(s) 11

3.1.2 Results and Discussion 12

3.1.3 Summary of Findings 21

3.2 Requirement Specifications 22

3.2.1 Functional Requirements 23

3.2.2 Non-functional Requirements 24

3.2.3 System Features 25

3.2.4 Actors 26

3.2.5 Requirements Justification 26

3.3 System Modelling 28

3.3.1 Use Case Diagram 28

3.3.2 Use Case Descriptions 29

**CHAPTER 4 SYSTEM DESIGN 34**

4.1 System Design 34

4.2 System Architecture 34

4.3 Interface Design 35

4.4 Database Design 41

4.4.1 Entity Relationship Diagram (ERD) 41

4.4.2 Data Dictionary 42

**CHAPTER 5 46**

5.1 CONCLUSION 46

5.2 Implementation Plan - FYP2 46

**REFERENCES 48**

**APPENDIX A SAMPLE APPENDIX 1 49**

**APPENDIX B SAMPLE APPENDIX 2 50**

LIST OF TABLES

Table 2.1: Comparison between existing systems 9

Table 3.1: Functional Requirements of the System 23

Table 3.2: Non-Functional Requirements (NFRs) 24

Table 3.3: Use Case Description – REGISTER 29

Table 3.4: Use Case Description – LOGIN 30

Table 3.5: Use Case Description – Generate Project Ideas 30

Table 3.6: Use Case Description – Do Requirement Analysis 31

Table 3.7: Use Case Description – Generate UML Diagram 31

Table 3.8: Use Case Description – Recommend Database Schema 31

Table 3.9: Use Case Description – Generate Code 32

Table 3.10: Use Case Description – Review Code 32

Table 3.11: Use Case Description – Generate Test Cases 33

Table 3.12: Use Case Description – Manage & Export Files 33

Table 4.1: Data Dictionary for User Entity 42

Table 4.2: Data Dictionary for Project Entity 42

Table 4.3: Data Dictionary for Project_Idea Entity 42

Table 4.4: Data Dictionary for Requirement Entity 43

Table 4.5: Data Dictionary for UML_Diagram Entity 43

Table 4.6: Data Dictionary for DB_Table Entity 43

Table 4.7: Data Dictionary for DB_Field Entity 43

Table 4.8: Data Dictionary for Code_Artifact Entity 44

Table 4.9: Data Dictionary for Code_Review Entity 44

Table 4.10: Data Dictionary for Test_Case Entity 44

## LIST OF FIGURES

Figure 2.1: GitHub Copilot Logo 5

Figure 2.2: ChatGPT Logo 6

Figure 2.3: Ask SIA Logo 7

Figure 2.4: Visual Paradigm Logo 7

Figure 2.5: BlueJ Logo 8

Figure 3.1: Distribution of respondents by level of study 12

Figure 3.2: Students' familiarity level with the SDLC 13

Figure 3.3: Most challenging SDLC phases identified by respondents 14

Figure 3.4: Difficulty level in project idea generation 15

Figure 3.5: Challenges faced when writing software requirements 15

Figure 3.6: Difficulty level in designing UML diagrams 16

Figure 3.7: Difficulty level in coding and implementation 17

Figure 3.8: Difficulty level in testing and debugging 17

Figure 3.9: Confidence level in creating UML diagrams 18

Figure 3.10: Preferred AI features among respondents 18

Figure 3.11: Likelihood of using an AI assistant for SDLC tasks 19

Figure 3.12: Use Case Diagram for the AI-Powered Software Engineering Study Assistant 29

Figure 4.1: System Architecture 35

Figure 4.2: Login and Registration Interface 36

Figure 4.3: Main Dashboard Interface 36

Figure 4.4: Project Idea Generation Screen 37

Figure 4.5: Requirements Analysis Interface 37

Figure 4.6: UML Diagram Generator Interface 38

Figure 4.7: Database Schema Recommendation Screen 38

Figure 4.8: Code Generator Interface 39

Figure 4.9: Code Review Interface 39

Figure 4.10: Test Case Generation Interface 40

Figure 4.11: Export and File Management Interface 40

Figure 4.12: Entity Relationship Diagram 41

# LIST OF SYMBOLS

| | |
|---|---|
| A | Number of PLS or PCA components in the model |
| a | Number of the PLS or PCA component |
| b | PLS regression coefficient |
| b | Number of blocks (b=1,2,3.....,K) |
| C | Coarse APM block |
| $\boldsymbol{C}_p$ | Pooled covariance matrix for the two classes |

# LIST OF ABBREVIATIONS

| | |
|---|---|
| **AI** | Artificial Intelligence |
| **LLM** | Large Language Model |
| **NLP** | Natural Language Processing |
| **API** | Application Programming Interface |
| **UI** | User Interface |
| **UX** | User Experience |
| **DBMS** | Database Management System |
| **ML** | Machine Learning |
| **JSON** | JavaScript Object Notation |
| **HTML** | Hypertext Markup Language |
| **CSS** | Cascading Style Sheets |
| **JS** | JavaScript |
| **SQL** | Structured Query Language |
| **SSR** | Software System Requirements |
| **SDLC** | Software Development Life Cycle |
| **HTTP** | Hypertext Transfer Protocol |

# CHAPTER 1

# INTRODUCTION

## 1.1 Project Background

Software engineering students often encounter significant difficulties in mastering the full software development lifecycle (SDLC), from ideation to deployment. While theoretical understanding is typically achieved through coursework, many students struggle when translating abstract concepts into practical implementation. Key challenges include formulating innovative project ideas, analysing requirements, designing structured system models, and producing maintainable code that adheres to industry standards.

Traditional learning environments rely heavily on instructor feedback, peer collaboration, or online tutorials, all of which can be time-consuming, inconsistent, and limited in personalization. As class sizes grow and projects become more complex, students often find themselves without immediate access to contextual support during critical stages of software development.

The emergence of **Artificial Intelligence (AI)** and **Large Language Models (LLMs)** presents a new opportunity to provide real-time, adaptive guidance throughout the SDLC. By leveraging these models, an intelligent assistant can understand user prompts in natural language, generate project ideas, produce UML diagrams, recommend database schemas, perform code reviews, and even suggest automated test cases. Such an AI-powered system can serve as both a **mentor and collaborator**, helping software engineering students strengthen their technical and problem-solving skills through practice and feedback.

This project aims to harness these AI capabilities to create a comprehensive study assistant that not only supports the learning process but also encourages deeper understanding of the software engineering workflow, bridging the gap between theory and practice.

## 1.2 Problem Statement

Although a wide range of educational tools and online resources exist for programming and software design, most are specialized in isolated tasks such as code completion, UML generation, or syntax correction without providing a holistic, SDLC aware learning experience. Software engineering students, particularly those undertaking individual or capstone projects, frequently struggle to connect the various phases of development into a coherent process.

Students often begin with vague project ideas and limited understanding of how to translate requirements into robust design and implementation. They face difficulties visualizing architectures, generating test cases, and applying best practices in coding and documentation. Manual feedback from instructors, while valuable, cannot scale effectively to provide personalized and immediate guidance for every student.

Moreover, existing AI assistants like ChatGPT are trained for general use and lack the contextual sensitivity required for guiding students through software engineering workflows. They may produce technically correct outputs but often without alignment to SDLC principles, structured design methods, or educational objectives.

Hence, there is a clear need for a dedicated AI-based study assistant that integrates multiple phases of software engineering within a single, intelligent system. Such a platform should guide students step-by-step from project conception to testing while using fine-tuned LLMs to offer personalized, contextually relevant support that enhances both learning outcomes and software quality.

## 1.3 Objective

The overall objective of this project is to design and develop an AI-powered web-based assistant that supports software engineering students across the software development lifecycle. The specific objectives, structured around the SDLC methodology and LLM integration, are as follows:

1. To gather and analyse student needs, educational requirements, and common pain points across SDLC phases, forming the foundation for the system's functional and non-functional requirements.
2. To design the architecture and interface of the AI assistant, including modules for project ideation, requirements analysis, UML and database design, code review, and automated test case generation.
3. To develop the AI-powered study assistant as a web-based platform and integrate a fine-tuned Large Language Model (LLM) capable of understanding student inputs, generating structured software artefacts, and providing personalized feedback and coding recommendations.
4. To perform system testing for functionality, usability, and accuracy, and to evaluate the assistant's effectiveness in supporting learning and improving students' understanding of the software engineering process through pilot user testing and feedback.

## 1.4 Scope

The project encompasses the development of a web-based application that supports software engineering students through the major phases of the SDLC using AI-driven assistance. The main components include:

- **Project Idea Generation:** Suggesting and refining software project concepts.
- **Requirements Analysis:** Helping students formalize and structure requirements.
- **UML Diagram Generation:** Creating visual models (class, use case diagrams) from textual descriptions.
- **Database Schema Design:** Recommending relational models based on requirements.
- **Code Review and Suggestions:** Evaluating uploaded code for style, logic, and optimization.
- **Code Generation:** Automatically generating clean and structured source code templates based on defined requirements and selected technologies.
- **Automated Test Case Generation:** Producing basic test cases aligned with functional specifications.

The assistant will emphasize educational use rather than full industrial deployment. It will focus on usability, context-aware AI guidance, and educational alignment with software engineering principles. Integration with external IDEs or large-scale collaborative environments is beyond the project's scope but may be explored as future work. The users who are targeted by this project include the following:

- Primary Users: Undergraduate students majoring in Software Engineering, Computer Science, or Information Technology, particularly those engaged in programming assignments, capstone projects, or software development courses.

# CHAPTER 2

# LITERATURE REVIEW

This chapter examines current AI systems that support programming and software development. Its purpose is to evaluate their strengths, limitations, and identify the gap that motivates the development of the AI-Powered Software Engineering Study Assistant. While many AI tools assist with code writing or answering programming queries, most focus on isolated functions like code completion or conversational support. None provide a fully integrated intelligent platform that spans the entire SDLC while offering educational value.

## 2.1 Related Works

The systems reviewed include GitHub Copilot, ChatGPT, Ask SIA, Visual Paradigm, and BlueJ. Each excels in specific areas but falls short of delivering a comprehensive educational assistant that automates multiple SDLC phases.

### 2.1.1 GitHub Copilot

Figure 2.1: GitHub Copilot Logo

Developed collaboratively by GitHub and OpenAI, GitHub Copilot leverages large language models to provide real-time, context-aware code suggestions.

**Strengths:**

- Intelligent, context-sensitive code completion.
- Supports a wide range of programming languages.
- Seamless integration with popular IDEs like Visual Studio Code.

**Limitations:**

- Focused exclusively on code generation; lacks diagram and documentation capabilities.
- Does not incorporate bug detection or test case generation.
- Not designed as an educational support tool.

### 2.1.2 ChatGPT

Figure 2.2: ChatGPT Logo

ChatGPT is a conversational AI capable of generating and explaining code in multiple programming languages through natural language interaction.

**Strengths:**

- Understands natural language instructions effectively.
- Offers code explanations and debugging assistance.
- Supports conceptual learning and problem-solving.

**Limitations:**

- Does not automatically produce diagrams or test cases.
- Lacks integration with development environments.
- Output quality depends heavily on user prompts.

### 2.1.3 Ask SIA (Student Intelligent Assistant)

Figure 2.3: Ask SIA Logo

Ask SIA is an academic chatbot designed to support student learning by responding to technical and academic questions.

**Strengths:**

- Simple, user-friendly chat interface.
- Provides quick, informative answers.
- Encourages independent learning.

**Limitations:**

- Limited to text-based assistance; no code generation or analysis.
- Does not support diagram, documentation, or test generation.
- Not integrated with software engineering tools.

### 2.1.4 Visual Paradigm

Figure 2.4: Visual Paradigm Logo

Visual Paradigm is a professional software modeling tool used for manual creation of UML, ERD, and other software diagrams.

**Strengths:**

- Comprehensive diagramming capabilities.
- Supports software design, documentation, and team collaboration.

**Limitations:**

- Entirely manual operation requiring user expertise.
- Lacks AI-driven automation.
- Cannot generate or validate source code.

### 2.1.5 BlueJ

Figure 2.5: BlueJ Logo

BlueJ is an educational Java IDE tailored for beginners learning object-oriented programming.

**Strengths:**

- Simple and intuitive user interface.
- Visualizes class relationships to aid learning.

**Limitations:**

- No AI or automation features.
- Cannot intelligently generate or analyze code.
- Lacks diagram and test generation functionality.

## 2.2 Findings and Analysis

The review reveals that existing AI assistants and development environments focus on individual tasks such as code suggestion, syntax checking, or diagramming. None combines intelligent code generation, bug detection, diagram creation, test case generation, and automated documentation within a single platform. Furthermore, tools

like GitHub Copilot and ChatGPT primarily target professional developers, while BlueJ and Ask SIA focus on learners but lack automation.

Table 2.1: Comparison between existing systems

| System | Main Function | AI Used | Code Generation | Diagram Generation | Bug Detection | Test Case Creation | Documentation | Educational Focus |
|---|---|---|---|---|---|---|---|---|
| **GitHub Copilot** | Code completion | Yes | Yes | No | No | No | No | Limited |
| **ChatGPT** | Conversational code help | Yes | Yes | No | Partial | No | Yes | Yes |
| **Ask SIA** | Academic Q&A chatbot | Yes | No | No | No | No | No | Yes |
| **Visual Paradigm** | Manual UML tool | No | No | Yes (Manual) | No | No | Yes | Limited |
| **BlueJ** | Educational IDE | No | No | Partial (Visual) | No | No | No | Yes |
| **Proposed System** | AI-Powered Assistant | Yes | Yes (Auto) | Yes (Auto) | Yes | Yes | Yes | Student-oriented |

## 2.3 Summary of Findings

Existing tools either focus narrowly on coding (GitHub Copilot, ChatGPT), provide manual educational support (BlueJ, Ask SIA), or offer manual diagramming (Visual Paradigm). The AI-Powered Software Engineering Study Assistant addresses these shortcomings by combining intelligent automation with educational support across all SDLC phases—from requirements gathering to testing and documentation. This makes it a uniquely powerful tool for software engineering students.

## 2.4 Gap in Existing Systems

While AI assistants like GitHub Copilot and ChatGPT have revolutionized code writing, they lack comprehensive coverage of the SDLC and do not sufficiently support educational objectives. Conversely, educational tools such as BlueJ and Ask SIA lack automation and remain manual.

The proposed AI-Powered Software Engineering Study Assistant bridges this divide by integrating:

- AI-driven automation for practical software development activities.
- Educational guidance to deepen students' understanding of each SDLC phase.

By merging these capabilities into a unified intelligent platform, this system not only streamlines software development but also enriches the learning journey for software engineering students.

In conclusion, this chapter has reviewed several existing AI and educational systems and identified a gap for an integrated AI-based assistant that supports all SDLC phases. The next chapter presents the system requirements and design of the proposed AI-Powered Software Engineering Study Assistant.

# CHAPTER 3

# REQUIREMENT ANALYSIS

## 3.1 Requirements Elicitation

Requirements elicitation is the process of gathering information from users to understand their needs, challenges, and expectations toward the proposed **AI-Powered Software Engineering Study Assistant**. This process ensures that the system is developed based on real user problems rather than assumptions. Through requirements elicitation, the project team identifies the functional and non-functional requirements that will guide the development of the AI assistant.

For this project, requirements elicitation focuses on understanding the difficulties students face across different phases of the Software Development Life Cycle (SDLC) including project idea generation, requirements analysis, UML modelling, database design, coding, debugging, and testing. The feedback gathered from students helps ensure that the proposed AI assistant provides targeted support that aligns with their actual academic needs.

To obtain accurate and reliable information, two elicitation techniques were used: an online questionnaire and short semi-structured interviews. These techniques combine both quantitative and qualitative data, enabling a deeper understanding of student challenges and expectations.

### 3.1.1 Elicitation Technique(s)

**Technique 1: Questionnaire (Google Form)**

An online questionnaire was distributed to students from Software Engineering, Computer Science, and Information Technology programmes at Universiti Tenaga Nasional (UNITEN). The questionnaire consisted of multiple-choice, Likert-scale, and open-ended questions designed to explore the challenges students face in SDLC phases such as project idea generation, requirements analysis, UML modelling, database design, coding, debugging, and testing.

The questionnaire link was shared through student groups and peer networks. A total of **37 responses** were collected. All names were removed in this report to maintain participant privacy.

**Technique 2: Short Semi-Structured Interviews**

To complement the questionnaire findings, short interviews were conducted with a small group of students. Each interview lasted approximately 5–10 minutes and included open-ended questions that allowed participants to elaborate on their experiences and expectations. The interviews provided deeper insights into the specific difficulties students encounter during SDLC tasks and helped validate the survey results. This method also helped identify additional requirements that may not surface through the questionnaire alone**.**

### 3.1.2 Results and Discussion

#### 3.1.2.1 Survey Results

A total of **36 students** participated in the online questionnaire. The survey responses provided quantitative insights into students' familiarity with SDLC, the challenges they face in different development phases, and their interest in AI-powered support tools. The key results are summarised below.

**Current Level of Study (Q1)**

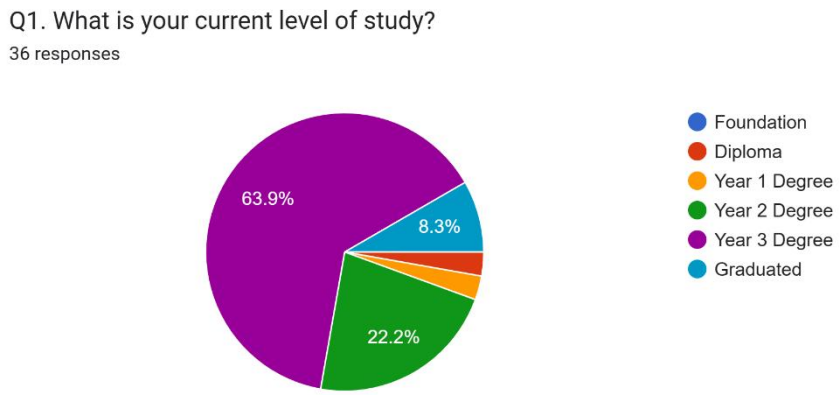


Figure 3.1: Distribution of respondents by level of study

A total of **36 students** participated in the survey. The majority of respondents were **Year 3 Degree students (23 students, 63.9%)**, followed by **Year 2 Degree students (8 students, 22.2%)**. A smaller percentage consisted of **Diploma students (1 student, 2.8%)**, **Year 1 Degree students (1 student, 2.8%)**, and **Graduate students (3 students, 8.3%)**.

This distribution suggests that most participants are senior students who have already completed SDLC-related coursework and project assignments. Therefore, their responses provide reliable insights into the real challenges faced during software engineering tasks.

### Familiarity with the SDLC (Q2)

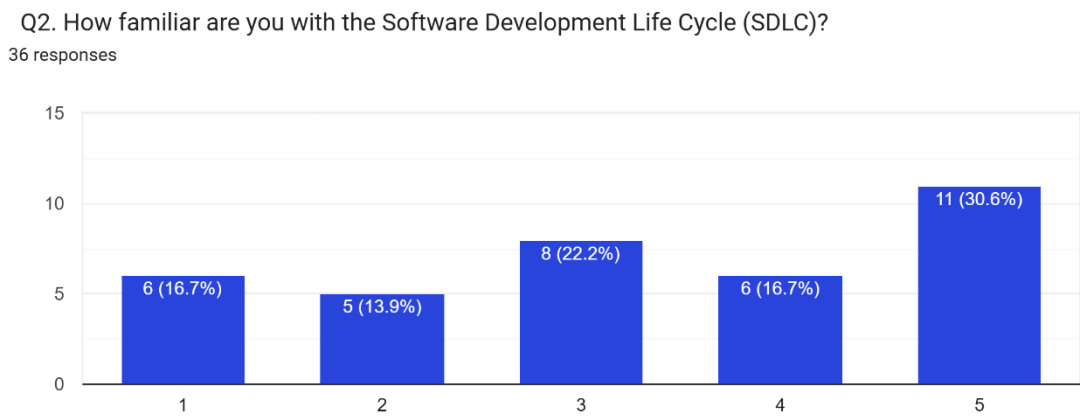


Figure 3.2: Students' familiarity level with the SDLC

Students rated their familiarity with the Software Development Life Cycle (SDLC) on a scale from **1 (not familiar)** to **5 (very familiar)**. The results were:

- **6 students (16.7%)** selected Level 1
- **5 students (13.9%)** selected Level 2
- **8 students (22.2%)** selected Level 3
- **6 students (16.7%)** selected Level 4
- **11 students (30.6%)** selected Level 5

Although a portion of respondents reported very high familiarity with SDLC concepts, the majority fall between **Levels 2 and 4**, indicating only **moderate understanding**. This suggests that many students still experience difficulties when applying SDLC in practice, especially during tasks such as requirements analysis, UML modelling, coding, debugging, and testing. This further confirms that an AI-powered assistant that can provide step-by-step guidance across the SDLC phases.

### Q3. Which SDLC phase do you find the most challenging?

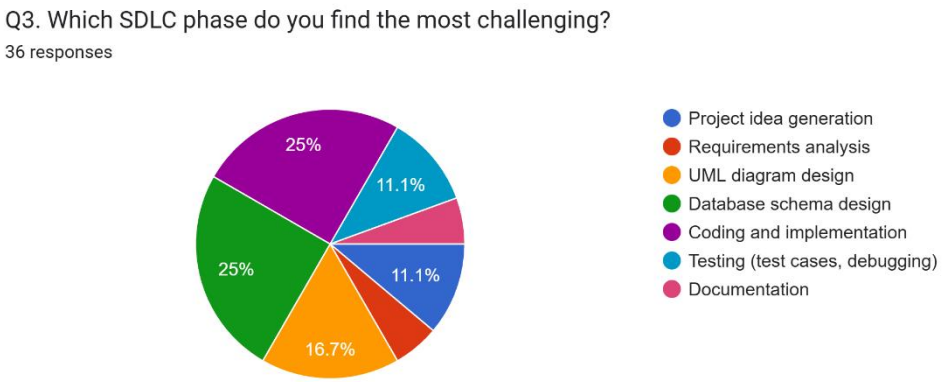


Figure 3.3: Most challenging SDLC phases identified by respondents

Respondents were asked to identify the SDLC phase they find most challenging. The responses show clear variation across different tasks:

- **Database schema design** – most frequently selected
- **Coding and implementation** – commonly reported
- **Testing (test cases, debugging)** – significant number
- **UML diagram design** – notable difficulty
- **Project idea generation** – selected by several students
- **Requirements analysis and documentation** – also mentioned

The results indicate that students struggle with **both early-phase design tasks** (idea generation, UML modelling, requirements analysis) and **later implementation tasks** (coding, debugging, testing). This reinforces the need for an AI assistant that supports learners **throughout the entire SDLC**, not only in coding.

**Q4. How difficult is it for you to come up with a project idea?**

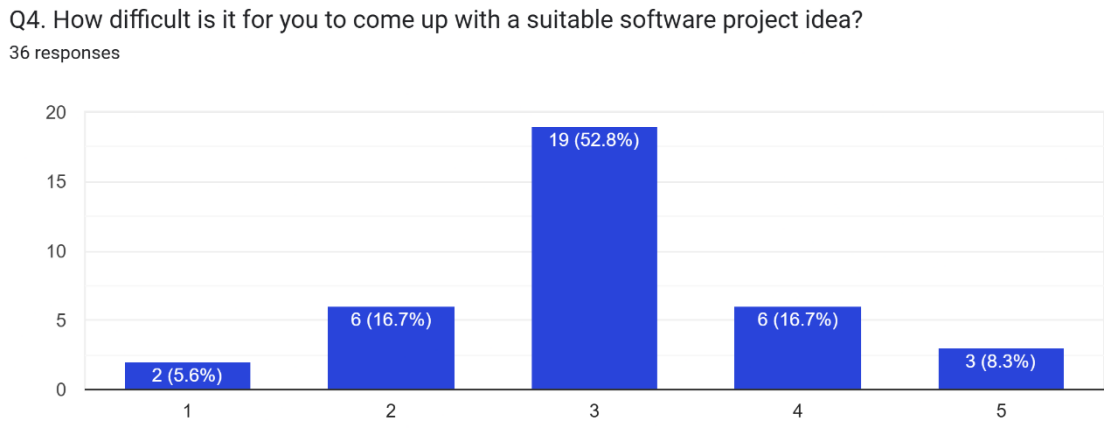


Figure 3.4: Difficulty level in project idea generation

Students rated their difficulty in generating project ideas on a scale of 1 to 5. The responses show a concentration between **levels 3 and 4**, indicating **moderate to high difficulty** in coming up with feasible and innovative ideas.

This suggests that many students struggle with the initial stage of the SDLC — project conceptualization. Hence, integrating an **AI-powered idea suggestion module** can help students explore practical and relevant project topics more efficiently.

### Q5. When writing software requirements, which part do you struggle with the most?

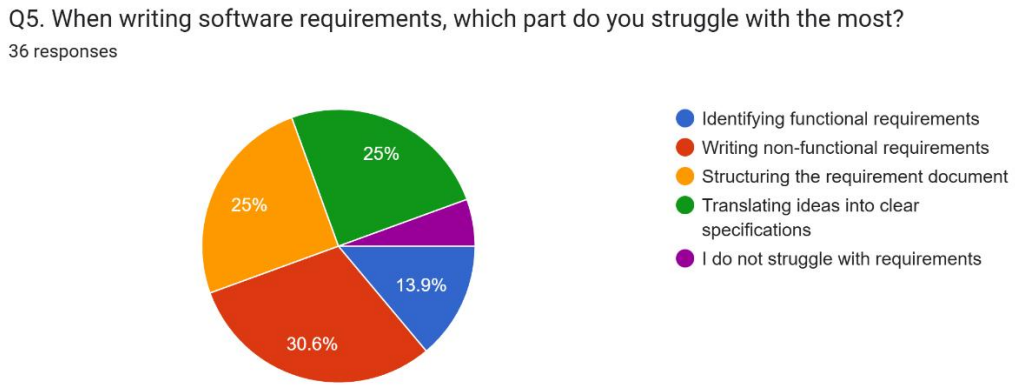


Figure 3.5: Challenges faced when writing software requirements

Students highlighted several challenges in writing software requirements, including:

- **Writing non-functional requirements**
- **Identifying functional requirements**
- **Structuring the requirements document**

- **Translating ideas into clear specifications**

These findings show that requirement engineering remains a major difficulty for students. This supports the inclusion of a **requirements refinement tool** in the AI system to help students structure and improve their requirements.

**Q6. How difficult is UML diagram design for you?**

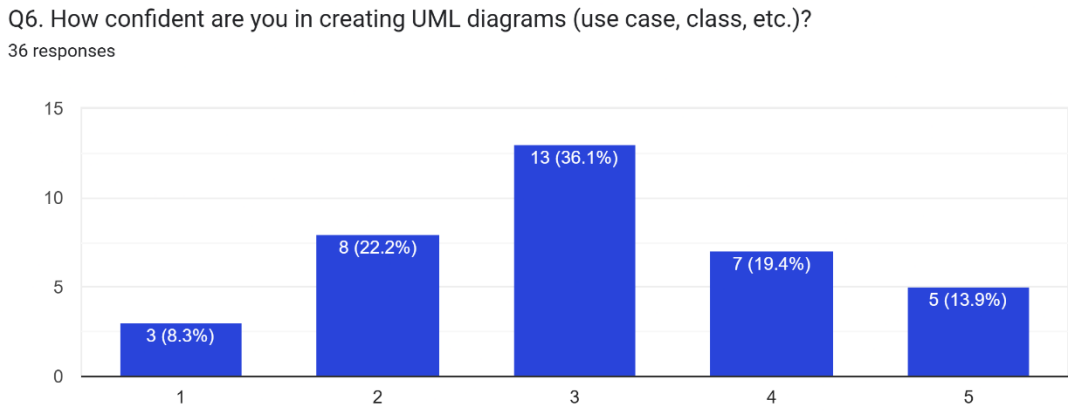


**Figure 3.6: Difficulty level in designing UML diagrams**

Students rated UML design difficulty on a scale of 1 to 5:

- Many students rated UML difficulty between **3 and 5**,
  indicating that UML diagrams — such as use-case diagrams, class diagrams, and activity diagrams — remain challenging.

This confirms that an **AI-driven UML diagram generator** would support students in visualizing system architecture correctly.

**Q7. How difficult is coding and implementation for you?**

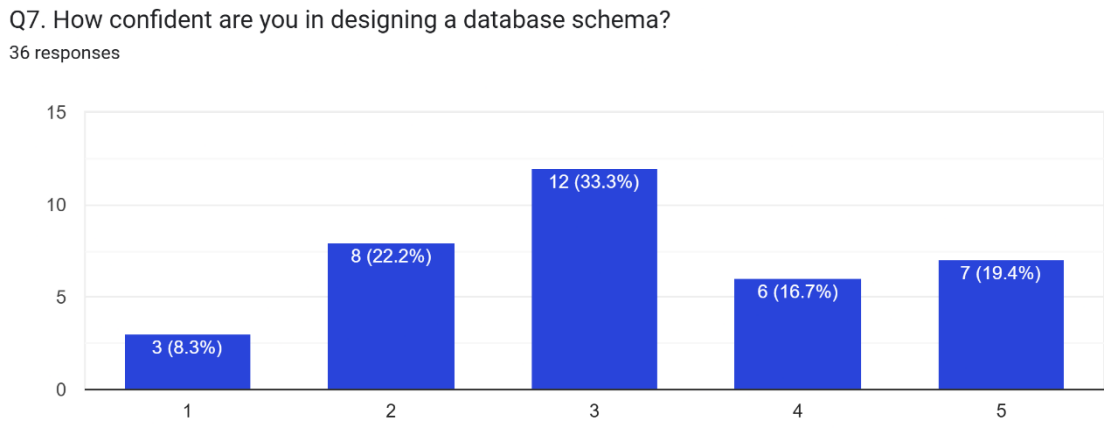


Figure 3.7: Difficulty level in coding and implementation

Responses ranged across the scale, but many students reported **moderate difficulty** (levels 2–4).
Students often struggle with:

- Logic errors
- Understanding error messages
- Applying correct programming structures

This indicates the need for AI-assisted code review and debugging features.

### Q8. How difficult is testing (test cases, debugging)?

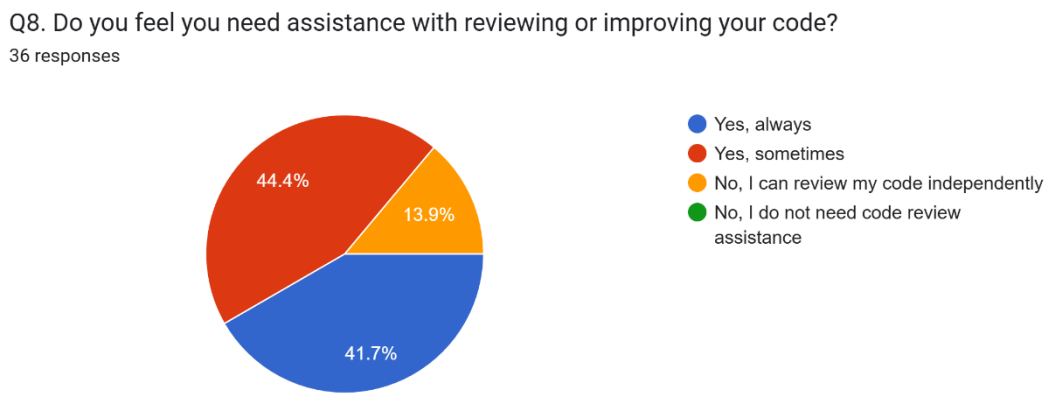


Figure 3.8: Difficulty level in testing and debugging

Many students rated testing and debugging as **moderately difficult**, with some selecting the highest difficulty levels (4 and 5). Testing is often one of the weakest areas in student projects, as it requires analytical thinking and attention to detail.

This highlights the importance of features such as **automatic test case generation** and **debugging guidance**.

### Q9. How confident are you in creating UML diagrams?

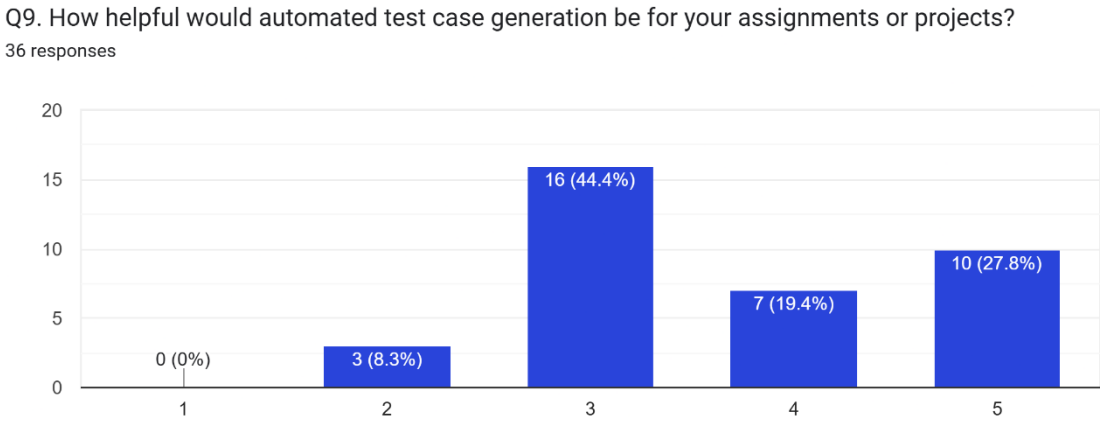


Figure 3.9: Confidence level in creating UML diagrams

Most responses are within **levels 2–4**, showing **low to moderate confidence** in UML diagram creation. This aligns with earlier findings and further supports creating an AI tool that can generate UML diagrams automatically and help students learn correct modelling practices.

### Q10. Which AI features would you find most useful?

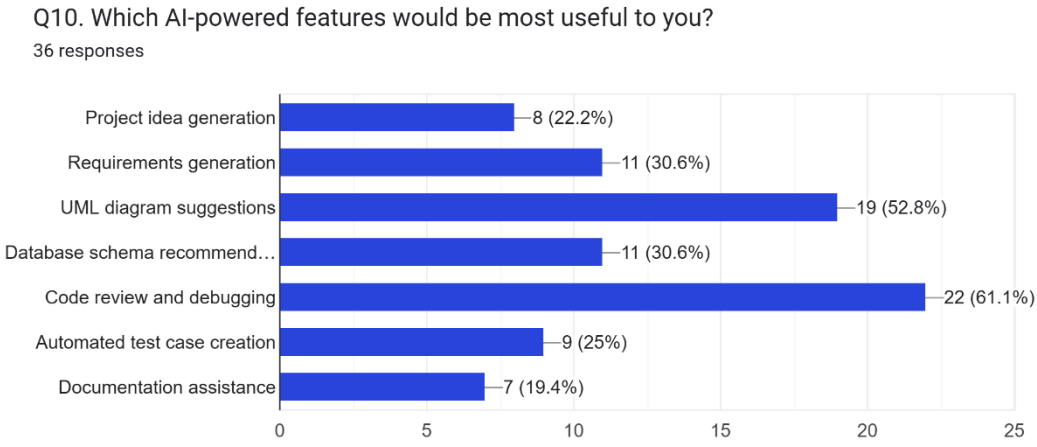


Figure 3.10: Preferred AI features among respondents

The most highly requested AI features include:

- UML diagram generation
- Requirements analysis assistance

- Debugging and code review
- Project idea generation
- Database schema recommendations
- Automated test case generation

This indicates that students prefer tools that support **multiple SDLC phases**, not just programming.

**Q11. How likely are you to use an AI assistant for SDLC tasks?**

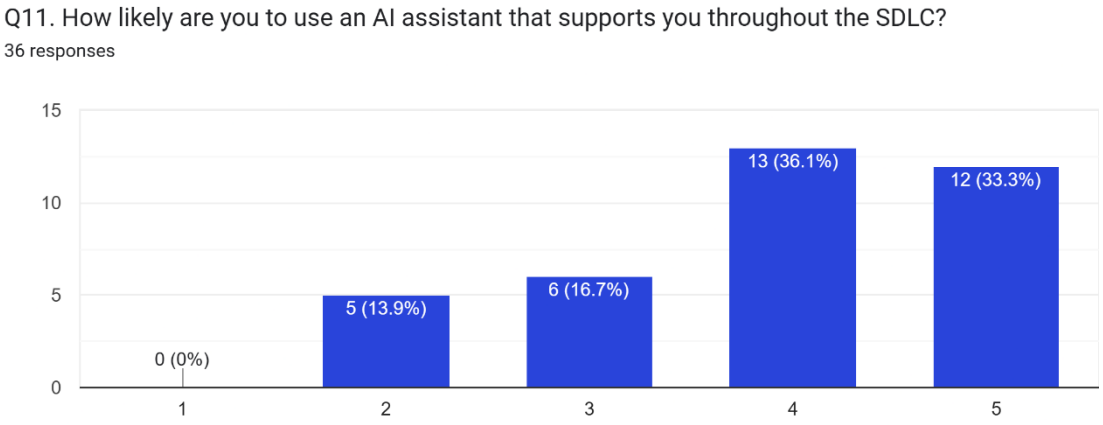


Figure 3.11: Likelihood of using an AI assistant for SDLC tasks

Most students selected **levels 4 and 5**, showing a **strong likelihood** of using an AI-powered assistant for SDLC tasks. This demonstrates high user acceptance and interest.

**Q12. What is your biggest challenge in software engineering? (Open-ended)**

Students provided a variety of open-ended responses. Common themes include:

- **Debugging and logic errors**
- **Designing the system architecture**
- **Converting ideas into requirements**
- **Managing time and workload**
- **Understanding database relationships**

These qualitative responses match the earlier quantitative findings and provide deeper insight into student difficulties.

#### 3.1.2.2 Interview Results

To complement the insights obtained from the questionnaire, short semi-structured interviews were conducted with three Year 3 Computer Science students. These

interviews aimed to gather deeper qualitative insights into students' challenges when applying SDLC (Software Development Life Cycle) in academic projects. The focus was on understanding difficulties related to requirements analysis, documentation, testing, and basic implementation tasks, as well as identifying expectations toward an AI-powered SDLC assistant.

**Interview 1: Yousif Al-Janahi (Year 3)**

**Q1:** Which part of the SDLC do you usually struggle with the most?
**A:** I often struggle with understanding the requirements and breaking them into clear steps.

**Q2:** When you receive a new project, are the SDLC steps clear to you?
**A:** Not always. Sometimes I'm unsure which phase I should begin with.

**Q3:** Do you find it difficult to decide what needs to be tested in your project?
**A:** Yes, I'm not always sure which features or functions are important to test.

**Q4:** Do you have trouble knowing what information should be included in the documentation?
**A:** Yes, I often don't know what should be written in each section.

**Q5:** Do you feel confident following the SDLC phases in the correct order?
**A:** Not really. I sometimes mix phases or skip steps without realizing.

**Q6:** Would guidance from an AI assistant through each SDLC phase be helpful?
**A:** Yes, it would make it easier for me to understand what to do at every stage.

**Q7:** Would simple example code or templates generated by the assistant be useful?
**A:** Yes, it would help me understand how to begin implementation.

**Q8:** What do you expect an AI SDLC assistant to provide?
**A:** Clear explanations, structured steps, and small sample code to help me start correctly.

**Interview 2: Musab Alsakkaf (Year 3)**

**Q1:** Do you find it challenging to apply SDLC concepts in your university projects?
**A:** Yes, I understand the theory but applying it practically is difficult.

**Q2:** What is the most challenging part of the requirements phase for you?
**A:** Identifying the important requirements and organizing them correctly.

**Q3:** Do you struggle to understand what should be included in testing?
**A:** Yes, I'm often unsure which parts of the system need to be tested.

**Q4:** Is writing SDLC documentation difficult?
**A:** Yes, because I'm not always sure how detailed it should be.

**Q5:** Do you feel uncertain when transitioning between SDLC phases?
**A:** Yes, sometimes I don't know when to move to the next phase.

**Q6:** Would an AI assistant that explains each SDLC stage help you work more efficiently?
**A:** Definitely. It would reduce confusion and help me stay on track.

**Q7:** Would basic generated code snippets or templates help you start coding?
**A:** Yes, they would give me a clear starting point.

**Q8:** What is the most important feature you expect from an AI SDLC tool?
**A:** Helping me understand the task clearly and providing simple examples.

**Interview 3: Orier Mohamed (Year 3)**

**Q1:** When starting a new project, is organizing the SDLC phases difficult for you?
**A:** Yes, I'm often unsure how to distribute tasks across the phases.

**Q2:** Do you find it challenging to understand the overall SDLC process?
**A:** Yes, sometimes I understand the idea but not how the phases connect.

**Q3:** Is requirement analysis a difficult phase for you?
**A:** Yes, writing requirements clearly and in a structured format is challenging.

**Q4:** Do you struggle with knowing what information should go into the documentation?
**A:** Yes, I'm not always sure what is necessary or unnecessary.

**Q5:** Do you find it hard to identify which parts of your system should be tested?
**A:** Yes, I'm not always certain which features should be tested first.

**Q6:** Would a tool that explains SDLC phases in simple language help you?
**A:** Yes, it would make project work much easier.

**Q7:** Would code templates or small generated snippets help you start implementation?
**A:** Yes, they would help me move from the requirements to actual coding.

**Q8:** How could an AI SDLC assistant improve your workflow?
**A:** It would provide clarity, help with organization, and make both documentation and coding easier.

### 3.1.3 Summary of Findings

The data gathered from both the questionnaire and the interviews provide a clear picture of the challenges students face when working through different stages of the Software Development Life Cycle (SDLC). Overall, the feedback shows that students continue to

struggle with several core tasks and require more structured support to complete SDLC-related coursework.

One of the most consistent findings is that students have difficulty turning ideas into clear and structured requirements. Many respondents shared that they understand the general concepts of the SDLC, but translating these concepts into formal documentation is still challenging. This includes identifying functional and non-functional requirements, choosing the right format, and deciding what information should be included or removed.

The results also show that modelling tasks, especially UML diagrams and database schema design, remain a major pain point. Students commonly described UML diagrams as “confusing” or “time-consuming,” and survey responses confirm that confidence in this area is generally low. These difficulties are often linked to uncertainty about modelling rules and the lack of immediate feedback when diagrams are created manually.

Implementation-related challenges also appear frequently. Students mentioned struggling with understanding error messages, fixing logic issues, and writing clean, organised code. Testing was another area where many felt unsure, particularly when deciding what needs to be tested or how to write meaningful test cases.

A key takeaway from both the interviews and the survey responses is that students want more than just automated outputs. Many expressed the need for explanations that help them understand why a certain diagram, requirement, or test case is correct. This shows that students value guidance and learning support as much as automation.

Finally, many students indicated that an integrated AI-based tool would be helpful, particularly one that covers idea generation, requirement refinement, modelling, code review, and test case creation. The interest level in using such a tool was high, suggesting strong acceptance and real demand.

**Conclusion**
The findings clearly show that students need a system that can guide them through SDLC tasks, reduce confusion in complex areas such as modelling and testing, and strengthen their understanding through clear explanations. These insights directly support the design of the AI-Powered Software Engineering Study Assistant, ensuring that the planned features are well aligned with the actual needs of software engineering students.

## 3.2 Requirement Specifications

The AI-Powered SDLC Assistant is designed to support students during SDLC-related academic tasks by providing guided assistance in project ideation, requirements analysis, documentation, simple code generation, testing awareness, and basic debugging feedback. The following requirements define the expected behaviour and constraints of the system.

### 3.2.1 Functional Requirements

The functional requirements describe the core system behaviours that must be implemented to fulfil the objectives of the AI-Powered SDLC Assistant. These requirements were derived from questionnaire responses, interview findings, and an analysis of common challenges faced by Computer Science students when completing SDLC-related tasks. Table 3.X summarises the functional requirements of the proposed system.

Table 3.1: Functional Requirements of the System

| ID | Functional Requirement Description | Priority |
|---|---|---|
| **FR1** | The system shall allow users to create an account and log in so that their generated artefacts and project history can be saved and retrieved. | High |
| **FR2** | The system shall accept natural language input from users and generate relevant software project ideas based on keywords, domains, or problem descriptions. | High |
| **FR3** | The system shall assist users in converting vague or raw ideas into structured functional and non-functional requirement statements. | High |
| **FR4** | The system shall interpret textual descriptions to generate visual representations of basic UML diagrams (Use Case and Class diagrams) to serve as preliminary design models for the student. | High |
| **FR5** | The system shall recommend suitable database schema structures (e.g., tables, attributes, and simple relationships) based on the provided system requirements. | High |
| **FR6** | The system shall allow users to upload or paste code and provide automated feedback highlighting potential logical issues, errors, and opportunities for optimisation. | High |
| **FR7** | The system shall generate basic code templates or starter code structures based on user input, functional requirements, or module descriptions. | Medium |
| **FR8** | The system shall automatically generate simple test cases aligned with the system's functional requirements or user-provided function descriptions. | Medium |
| **FR9** | The system shall provide context-aware educational explanations for all generated outputs to support student learning and understanding of SDLC concepts. | Medium |

| | | |
|---|---|---|
| **FR10** | The system shall allow users to export generated artefacts including requirements, diagrams, and test cases, downloadable files such as PDF or image formats. | Medium |

The functional requirements listed above represent the essential capabilities needed to support students throughout various phases of the SDLC. These include ideation, requirements formulation, UML modelling, database schema design, code review, testing support, and educational guidance. Together, they form the foundation of the system's core functionality and ensure alignment with the project's objectives and scope.

### 3.2.2 Non-functional Requirements

The non-functional requirements define the quality attributes, constraints, and performance expectations of the AI-Powered SDLC Assistant. These requirements ensure that the system is usable, secure, reliable, and efficient for students throughout different SDLC tasks. Table 3.X summarises the non-functional requirements for the proposed system.

**Table 3.2: Non-Functional Requirements (NFRs)**

| ID | Non-Functional Requirements Description | Category |
|---|---|---|
| **NFR1** | The system should generate text responses and basic diagrams within 15 seconds to ensure a smooth user experience. | Performance |
| **NFR2** | The user interface shall be intuitive and suitable for undergraduate students, allowing new users to navigate core features without prior training. | Usability |
| **NFR3** | The system shall be accessible via modern web browsers (Chrome, Edge, Firefox, Safari) on both desktop and tablet devices without installation. | Compatibility |
| **NFR4** | User passwords shall be encrypted, and project data shall be stored securely to prevent unauthorized access. | Security |
| **NFR5** | The system should maintain high availability during normal usage periods to ensure consistent student access. | Reliability |
| **NFR6** | The system shall adopt modular architecture separating AI processing from the interface to support ease of maintenance and future expansion. | Maintainability |

The non-functional requirements outlined above ensure that the system performs efficiently, remains secure, and delivers a positive learning experience for students. These requirements

complement the functional requirements by defining the system's expected quality, usability, and operational constraints, thereby supporting the successful implementation of the AI-Powered SDLC Assistant.

### 3.2.3 System Features

The AI-Powered SDLC Assistant consists of several integrated modules designed to support Computer Science students across major phases of the Software Development Life Cycle. Each module aligns with the project scope and directly addresses challenges identified in the questionnaire and interviews. The main system features are described below.

1. **Project Idea Generation Module**

   This feature allows students to input natural language descriptions, keywords, or domains, and receive AI-generated software project ideas. The system helps students refine vague or unclear ideas into well-defined concepts suitable for academic assignments or FYP development.

2. **Requirements Analysis Module**

   This module assists users in transforming high-level or unclear ideas into structured functional and non-functional requirements. It guides students in understanding what should be included in requirement specifications, ensuring clarity and completeness.

3. **UML Diagram Generation Module**

   This feature interprets structured textual descriptions and produces basic UML diagrams such as Use Case and Class diagrams. These diagrams act as draft design models that help students visualise system behaviour and structure during early SDLC phases.

4. **Database Schema Recommendation Module**

   Based on the user's requirements, the system suggests suitable database schema designs, including tables, key attributes, and simple relationships. This supports students in understanding data modelling and preparing initial database structures.

5. **Code Review and Suggestion Module**

   This module allows students to upload or paste source code for analysis. The system provides automated feedback on potential logical issues, syntax errors, structural weaknesses, and opportunities for optimisation. This supports learning through guided improvement.

6. **Code Generation Module**

This module generates basic code templates or starter code structures based on user input, functional requirements, or module descriptions. It helps students begin implementation more efficiently by translating system requirements into initial executable code. This reduces the difficulty of starting new projects and improves understanding of how requirements map to actual code.

7. **Automated Test Case Generation Module**

   This feature generates basic test cases aligned with system requirements or specific function descriptions. It helps students understand how functional specifications translate into test scenarios, supporting early testing awareness in the SDLC.

8. **Educational Explanation Module**

   For every generated artefact (ideas, requirements, diagrams, tests, code feedback), the system provides context-aware explanations that help students learn *why* the output was produced. This positions the system as a learning assistant rather than only an automation tool.

9. **Export and Report Generation Module**

   This module enables users to download their generated artefacts—including requirements, diagrams, schemas, code reviews, and test cases—in formats such as PDF and image files. This supports documentation and reporting tasks required in academic submissions.

### 3.2.4 Actors

The key actors and stakeholders involved in the system are:

**Primary Actor**

1. **Student**
   - Use the system to obtain guidance on SDLC tasks.
   - Inputs project ideas, requirements, descriptions, or code for AI processing.

**System Actor**

2. **AI Engine (LLM)**
   - Processes user input and generates project ideas, diagrams, reviews, etc.

### 3.2.5 Requirements Justification

The requirements defined for the AI-Powered SDLC Assistant are justified based on the findings from the questionnaire, interviews, and the overall project scope. The goal of the

system is to support students in completing SDLC-related academic tasks more effectively using AI-driven assistance.

**1. Justification for Functional Requirements (FRs)**

The questionnaire and interview findings revealed that students face significant challenges across early and mid-stage SDLC activities, including generating project ideas, refining requirements, producing UML diagrams, designing database schemas, reviewing code, and understanding how to construct test cases. Many also expressed the need for contextual explanations to support learning. Therefore, each functional requirement is justified as follows:

- **FR1 (User Account & Project Saving)** is justified because students require a secure way to save progress, retrieve previous artefacts, and maintain continuity across different SDLC tasks.
- **FR2 and FR3 (Project Idea Generation & Requirements Refinement)** are justified because students indicated difficulty forming clear, structured project concepts and translating vague ideas into formal requirements.
- **FR4 and FR5 (UML Diagram Generation & Database Schema Recommendation)** are justified since students reported lacking confidence in modelling activities, particularly when converting requirements into design artefacts.
- **FR6 (Code Review and Error Detection)** is justified because students struggle to identify logical errors, improve coding practices, and understand code quality issues without instructor assistance.
- **FR7 (Code Generation)** is justified because many students have difficulty starting implementation from scratch. Generating starter code templates helps them bridge the gap from design to coding more effectively.
- **FR8 (Test Case Generation)** is justified because students often do not know what should be tested, how to structure test cases, or how functional requirements translate into testing scenarios.
- **FR9 (Educational Explanations)** is essential because the system is intended as a **learning assistant**, not merely an automation tool. Students explicitly requested explanations that help them understand *why* an artefact was generated.
- **FR10 (Exporting Artefacts)** is justified because students need to download requirements, diagrams, schemas, code reviews, and test cases for inclusion in academic assignments, reports, and FYP submissions.

**2. Justification for Non-Functional Requirements (NFRs)**

The Non-Functional Requirements ensure that the system performs reliably within an academic environment and provides a smooth learning experience:

- **Performance (NFR1)** is critical because long delays disrupt the learning flow, especially when generating diagrams or analysing code.
- **Usability (NFR2)** is justified since undergraduate students prefer simple, intuitive interfaces without complex navigation.
- **Compatibility (NFR3)** is necessary because the system must run on common browsers (Chrome, Edge, Firefox, Safari) used across laptops and tablets.
- **Security (NFR4)** is required to protect accounts, stored artefacts, and sensitive project information.
- **Reliability (NFR5)** ensures the system is consistently available during coursework periods, preventing interruptions during assignments or study sessions.
- **Maintainability (NFR6)** is important so the system can be updated easily, especially as AI models evolve or new features are added.

**3. Alignment With Project Scope**

All defined requirements align directly with the project's scope and intended purpose:

- Supporting students throughout SDLC learning,
- Providing AI-driven guidance and scaffolded learning support,
- Automating the generation of structured artefacts (ideas, requirements, diagrams, schemas, reviews, test cases, code templates),
- Enhancing understanding of SDLC concepts rather than replacing academic work.

Therefore, the functional and non-functional requirements are **necessary, feasible, and fully aligned** with the goals of the AI-Powered SDLC Assistant.

## 3.3 System Modelling

This section presents the system models developed for the AI-Powered Software Engineering Study Assistant. System modelling is used to describe how users interact with the system, how processes flow internally, and how different system components are structured. The models developed support the requirements gathered in Chapter 3 and provide a visual representation of the system's functional behavior.

The system models included in this section are:

- Use Case Diagram
- Use Case Descriptions

### 3.3.1 Use Case Diagram

Figure 3.12 shows the Use Case Diagram for the AI-Powered Software Engineering Study Assistant. The diagram illustrates how the student interacts with various system functions

including login, idea generation, requirements analysis, UML diagram generation, database schema recommendation, code generation, code review, test case creation, and export management. The AI Engine acts as a supporting system actor that processes student requests and generates the required outputs.

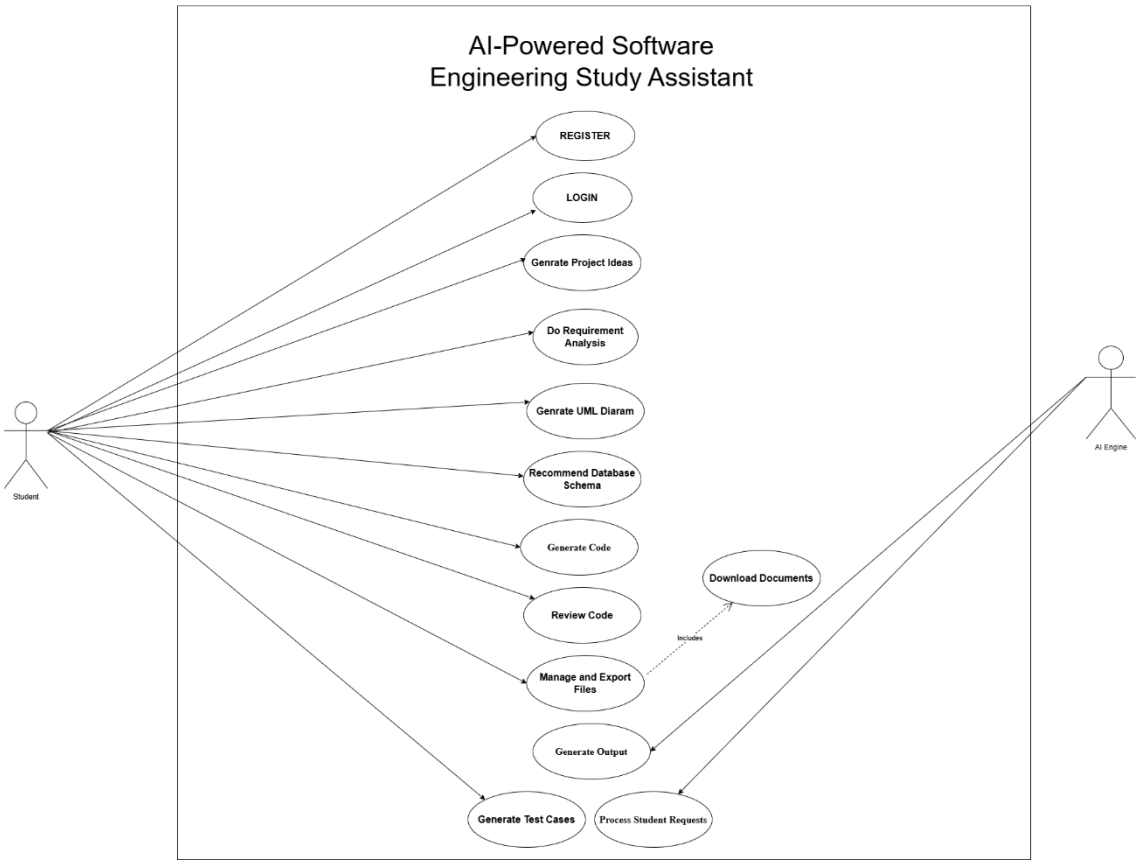


Figure 3.12: Use Case Diagram for the AI-Powered Software Engineering Study Assistant

### 3.3.2 Use Case Descriptions

The following tables summarise the main use cases that describe how each function behaves from the student's perspective.

**Use Case: REGISTER**

Table 3.3: Use Case Description – REGISTER

| Item | Description |
|---|---|
| Use Case Name | REGISTER |
| Actor | Student |

| | |
|---|---|
| **Goal** | **Create a new user account.** |
| **Precondition** | **User is not registered.** |
| **Main Flow** | **1. Student enters name, email, password.2. System validates input.3. System creates account.** |
| **Postcondition** | **New account stored in the system.** |
| **Exception** | **Email already exists; invalid password.** |

**Use Case: LOGIN**

Table 3.4: Use Case Description – LOGIN

| **Item** | **Description** |
|---|---|
| **Use Case Name** | **LOGIN** |
| **Actor** | **Student** |
| **Goal** | **Authenticate and access system dashboard.** |
| **Precondition** | **User has a registered account.** |
| **Main Flow** | **1. Student enters email & password.2. System verifies credentials.3. Students accesses dashboard.** |
| **Postcondition** | **User session is active.** |
| **Exception** | **Wrong password: account not found.** |

**Use Case: Generate Project Ideas**

Table 3.5: Use Case Description – Generate Project Ideas

| **Item** | **Description** |
|---|---|
| **Use Case Name** | **Generate Project Ideas** |
| **Actor** | **Student** |
| **System Actor** | **AI Engine** |
| **Goal** | **Provide AI-generated software project ideas.** |
| **Precondition** | **Student is logged in.** |
| **Main Flow** | **1. Student enters keywords.2. AI processes input.3. AI returns project ideas.** |
| **Postcondition** | **List of suggested project ideas displayed.** |
| **Exception** | **Empty or unclear input.** |

**Use Case: Do Requirement Analysis**

Table 3.6: Use Case Description – Do Requirement Analysis

| Item | Description |
|---|---|
| Use Case Name | Do Requirement Analysis |
| Actor | Student |
| System Actor | AI Engine |
| Goal | Convert raw idea into structured FR & NFR. |
| Precondition | Student provides initial idea. |
| Main Flow | 1. Student inputs rough description.2. AI analyses content.3. AI outputs structured requirements. |
| Postcondition | Requirements stored under the project. |
| Exception | Insufficient detail provided. |

**Use Case: Generate UML Diagram**

Table 3.7: Use Case Description – Generate UML Diagram

| Item | Description |
|---|---|
| Use Case Name | Generate UML Diagram |
| Actor | Student |
| System Actor | AI Engine |
| Goal | Generate Use Case or Class diagram description. |
| Precondition | Student enters system behaviour or interactions. |
| Main Flow | 1. Student enters description.2. AI interprets input.3. AI outputs UML structure. |
| Postcondition | UML diagram data saved. |
| Exception | Missing or unclear system details. |

**Use Case: Recommend Database Schema**

Table 3.8: Use Case Description – Recommend Database Schema

| Item | Description |
|---|---|

| | |
|---|---|
| **Use Case Name** | **Recommend Database Schema** |
| **Actor** | **Student** |
| **System Actor** | **AI Engine** |
| **Goal** | **Suggest tables, attributes, and relationships.** |
| **Precondition** | **Requirements are available.** |
| **Main Flow** | **1. Student inputs requirements.2. AI analyses entities.3. AI outputs schema suggestion.** |
| **Postcondition** | **Recommended schema displayed and saved.** |
| **Exception** | **Incomplete requirements.** |

**Use Case: Generate Code**

Table 3.9: Use Case Description – Generate Code

| **Item** | **Description** |
|---|---|
| **Use Case Name** | **Generate Code** |
| **Actor** | **Student** |
| **System Actor** | **AI Engine** |
| **Goal** | **Produce starter code for modules or functions.** |
| **Precondition** | **Student provides module description.** |
| **Main Flow** | **1. Student enters description.2. AI generates code template.3. System displays formatted code.** |
| **Postcondition** | **Code available for download.** |
| **Exception** | **Unsupported language or unclear description.** |

**Use Case: Review Code**

Table 3.10: Use Case Description – Review Code

| **Item** | **Description** |
|---|---|
| **Use Case Name** | **Review Code** |
| **Actor** | **Student** |
| **System Actor** | **AI Engine** |
| **Goal** | **Provide debugging and improvement suggestions.** |

| | |
|---|---|
| **Precondition** | **Student uploads valid code.** |
| **Main Flow** | **1. Student submits code.2. AI analyses logic and style.3. AI returns issues & recommendations.** |
| **Postcondition** | **Review report stored.** |
| **Exception** | **Invalid or unreadable code submitted.** |

**Use Case: Generate Test Cases**

Table 3.11: Use Case Description – Generate Test Cases

| **Item** | **Description** |
|---|---|
| **Use Case Name** | **Generate Test Cases** |
| **Actor** | **Student** |
| **System Actor** | **AI Engine** |
| **Goal** | **Produce test cases based on requirements or functions.** |
| **Precondition** | **Requirements/function description exists.** |
| **Main Flow** | **1. Student inputs requirement/function.2. AI generates test cases.3. System displays test cases.** |
| **Postcondition** | **Test cases saved to project.** |
| **Exception** | **Insufficient test information.** |

**Use Case: Manage and Export Files**

Table 3.12: Use Case Description – Manage & Export Files

| **Item** | **Description** |
|---|---|
| **Use Case Name** | **Manage & Export Files** |
| **Actor** | **Student** |
| **Goal** | **Export SDLC artefacts (PDF, text, images).** |
| **Precondition** | **Artefact exists in project.** |
| **Main Flow** | **1. Student selects artefact.2. Student chooses export format.3. System generates file.** |
| **Postcondition** | **File downloaded or stored.** |
| **Exception** | **Export failure.** |

# CHAPTER 4

# SYSTEM DESIGN

## 4.1 System Design

The purpose of this chapter is to present the overall design of the AI-Powered Software Engineering Study Assistant. The system design explains how the solution is structured, how its components communicate, and how information flows between the user interface, backend services, AI engine, and the Firebase database.

The system design for this project is divided into three main parts:

1. **System Architecture** explains the high-level structure of the system and illustrates how the user interface, backend server, AI processing engine, and Firebase services interact.
2. **Interface Design** presents the primary layout and design of the application screens, demonstrating how students interact with various SDLC assistance features.
3. **Database Design** defines how data is stored and organized using Firebase, including the entity structure and data dictionary.

Together, these design components provide a clear overview of how the proposed system operates and ensure that it satisfies the functional and non-functional requirements outlined in earlier chapters.

## 4.2 System Architecture

The system architecture describes how the AI-Powered SDLC Study Assistant is structured and how each component interacts to deliver intelligent SDLC-related guidance to students. The architecture is designed to ensure scalability, reliability, and efficient data processing, while supporting smooth communication between the student interface, backend processing layer, AI engine, and Firebase database services.

The system follows a client–server architecture, where the web-based client application sends requests to a backend server. The backend server manages authentication, request routing, AI processing, and data storage. The AI Engine, powered by a Large Language Model (LLM), processes natural-language requests and generates outputs such as project ideas, requirement refinements, UML diagram descriptions, database schema recommendations, debugging insights, and test cases.

When a student submits input, the request is first handled by the API Gateway, which validates the request and forwards it to the appropriate module inside the Application Server. The Application Server then communicates with the AI Engine to perform intelligent processing. Once processed, the AI response is returned to the user through the client application.

All generated outputs such as requirements, diagrams, reviews, and exported files are securely stored using Firebase services.
Firebase Authentication manages login and access control.
Firebase Firestore stores user data and generated artefacts.
Firebase Cloud Storage stores downloadable files and large generated outputs.

A high-level system architecture diagram, illustrating the interaction between the client layer, backend layer, AI engine, and data layer, is shown in Figure 4.1.

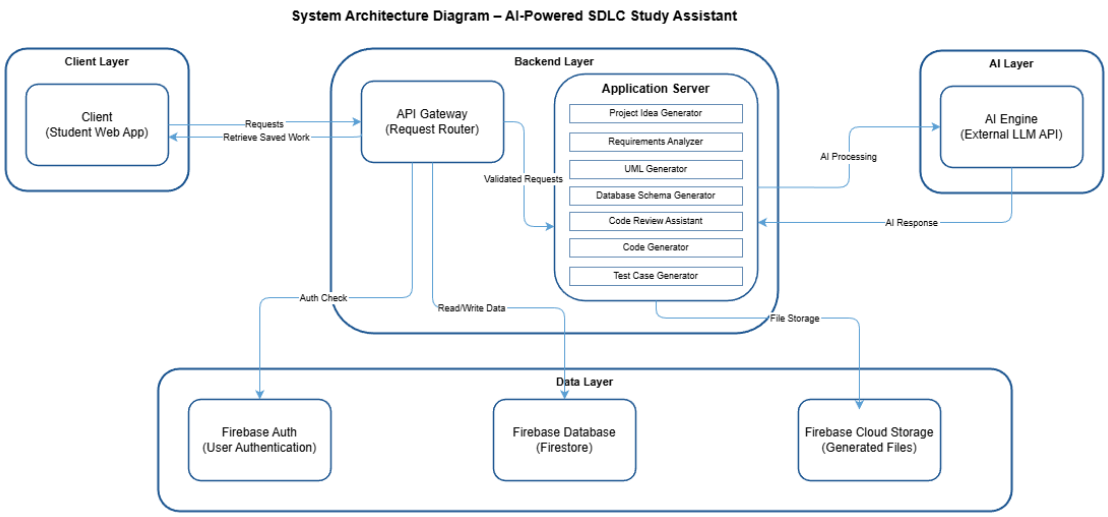


Figure 4.1: System Architecture

## 4.3 Interface Design

The interface design of the AI-Powered Software Engineering Study Assistant focuses on providing a clean, intuitive, and user-friendly environment for students to perform SDLC-related tasks efficiently. The design emphasizes simplicity and clarity to ensure accessibility for students with varying levels of technical experience. All features are organized into dedicated modules, and the interface maintains consistent navigation, typography, and layout to support ease of use.

Each interface screen is designed to guide students through specific tasks, such as generating project ideas, refining requirements, creating UML diagrams, designing database schemas, reviewing code, and generating test cases. The following subsections present the main interface screens of the system along with brief explanations of their purpose and functionality.

**Login and Registration Screen**

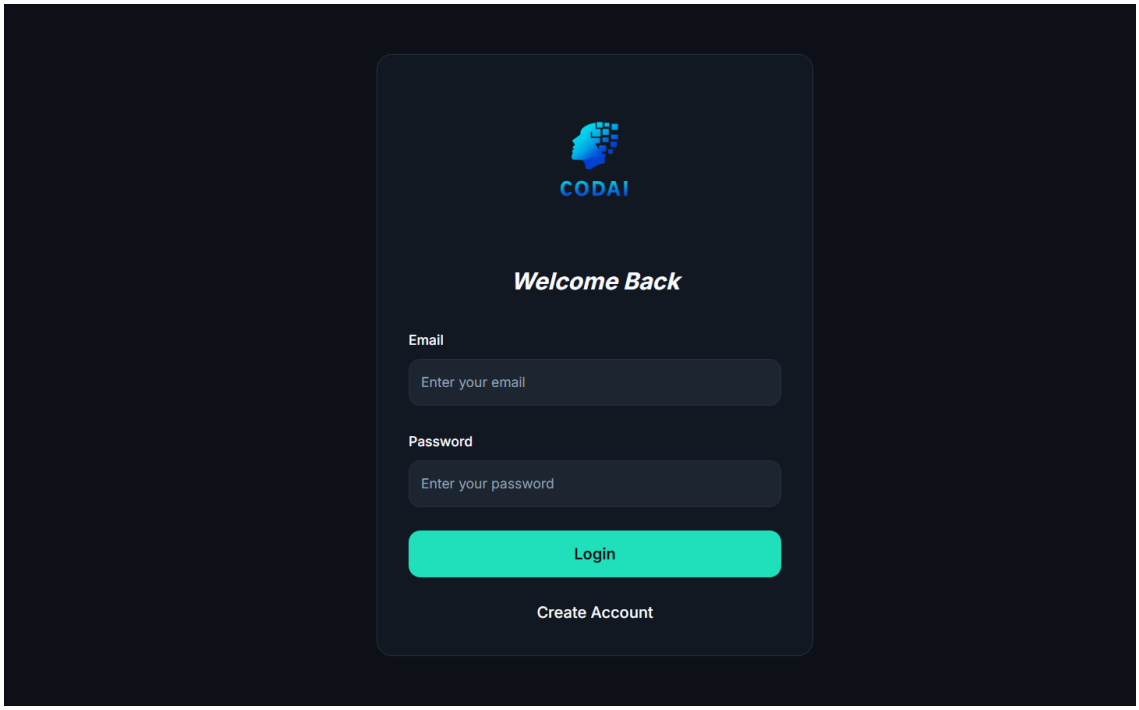


Figure 4.2: Login and Registration Interface

This screen allows users to create an account or log in to the system. It includes fields for email and password and is integrated with Firebase Authentication to ensure secure user access. Once authenticated, users are directed to the main dashboard.

### Dashboard Screen

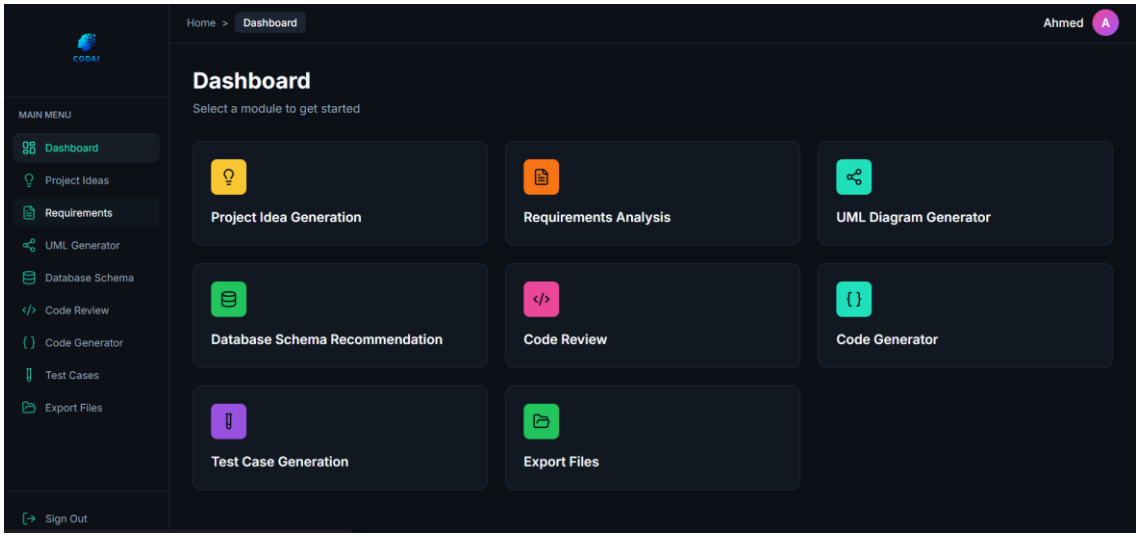


Figure 4.3: Main Dashboard Interface

The dashboard acts as the central navigation hub for the system. It displays all available AI-powered modules in a structured layout, allowing students to quickly access project idea generation, requirements analysis, UML generation, database schema suggestion, code review, test case generation, and file export features.

### Project Idea Generation Screen

Figure 4.4: Project Idea Generation Screen

This screen enables students to enter keywords, problem descriptions, or areas of interest. The AI processes the input and generates multiple software project ideas, each accompanied by brief explanations. This feature assists students in selecting feasible and relevant project topics.

### Requirements Analysis Screen

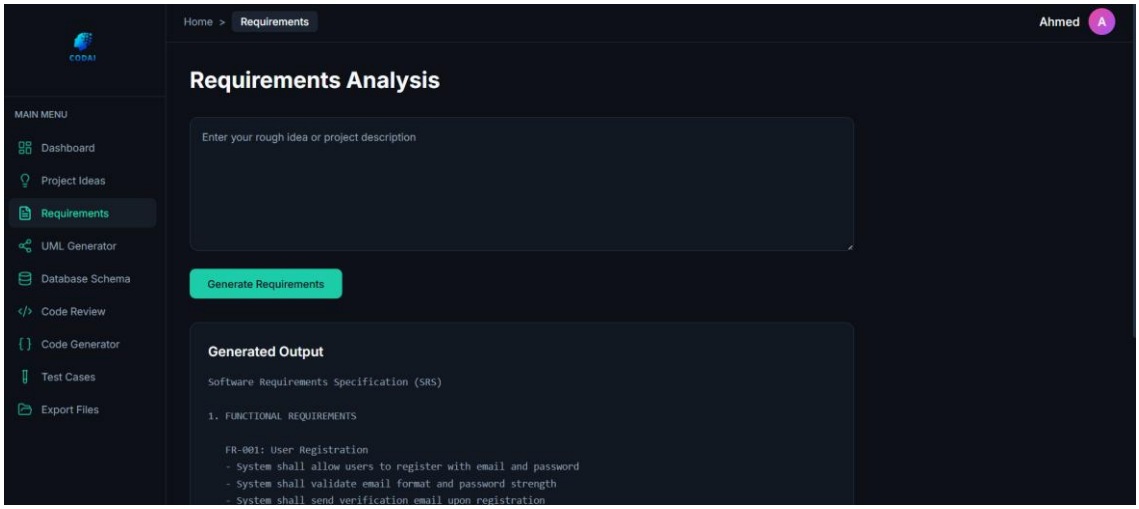


Figure 4.5: Requirements Analysis Interface

The Requirements Analysis screen allows students to input rough ideas or draft requirements. The AI refines the content into structured functional and non-functional requirements. This helps students learn proper formatting and ensures clarity and completeness in requirement documentation.

### UML Diagram Generator Screen

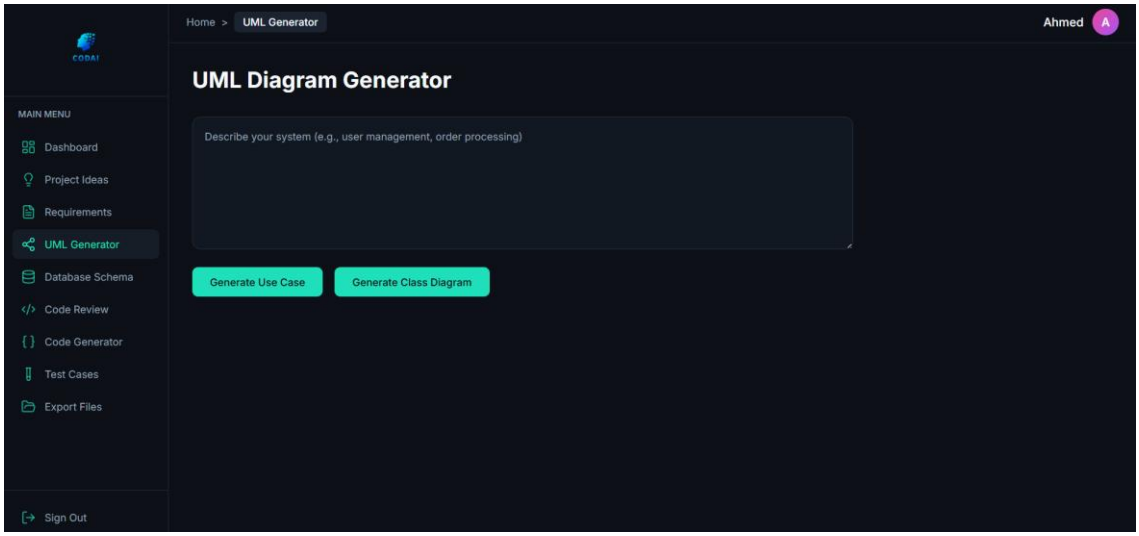


Figure 4.6: UML Diagram Generator Interface

Students can describe system interactions or behaviors using natural language. The AI interprets the input and generates structured UML elements such as actors and use cases. This assists students in visualizing preliminary system designs and reduces the difficulty of manual UML modelling.

### Database Schema Recommendation Screen

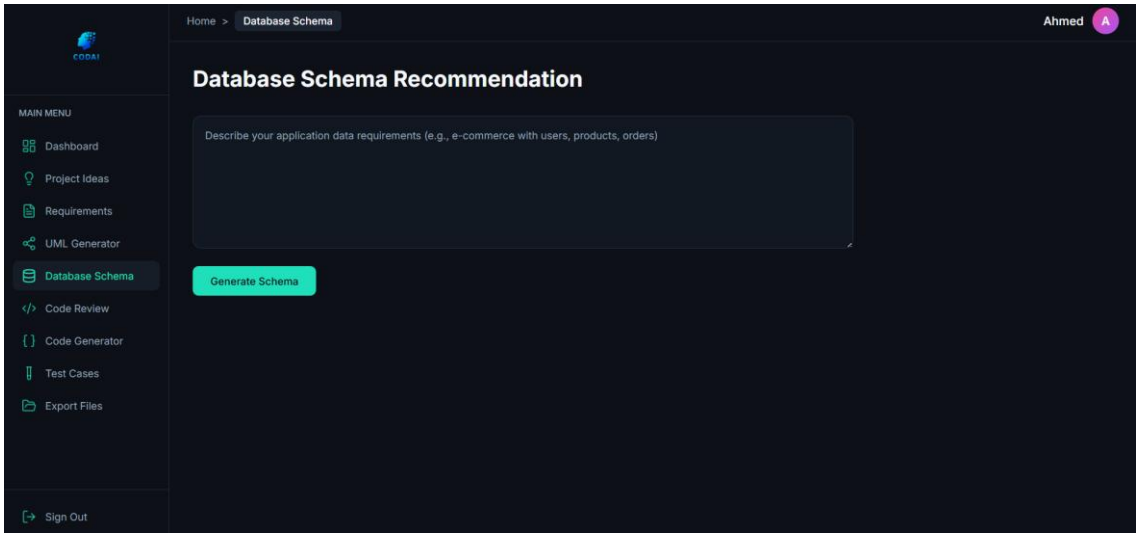


Figure 4.7: Database Schema Recommendation Screen

This screen analyzes user-provided requirements and suggests suitable database tables, attributes, and relationships. It helps students understand data modelling concepts and supports the design of initial database structures aligned with their project requirements.

### Code Generator Screen

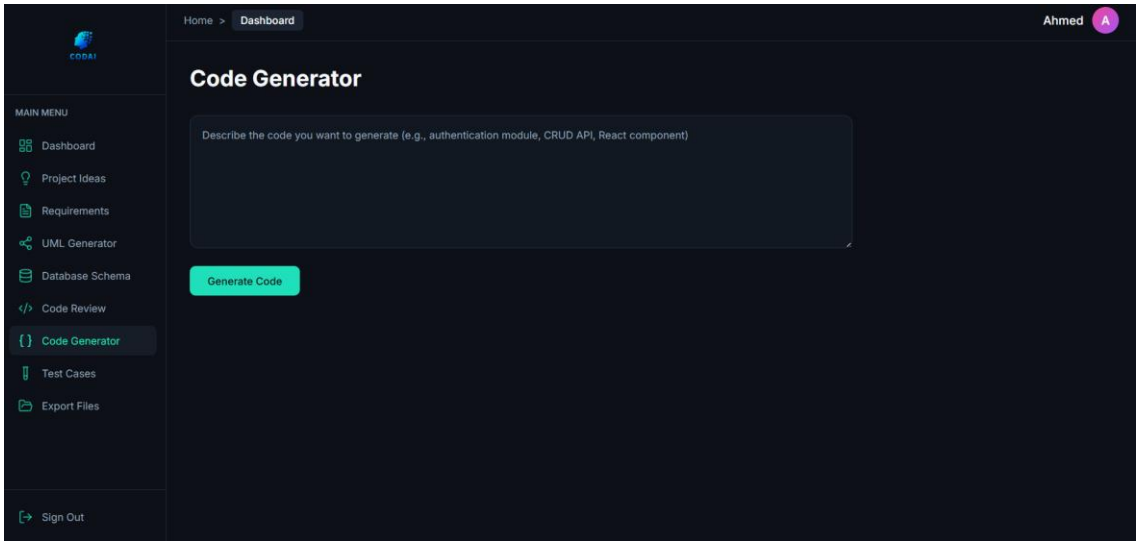


Figure 4.8: Code Generator Interface

This screen allows students to enter natural-language descriptions of modules or functionality. The AI engine generates basic starter code templates based on the provided input. The generated code is displayed in a formatted editor-style container for easy copying. This screen helps students transition from design to implementation by providing an initial code structure that they can extend in their IDE.

### Code Review Screen

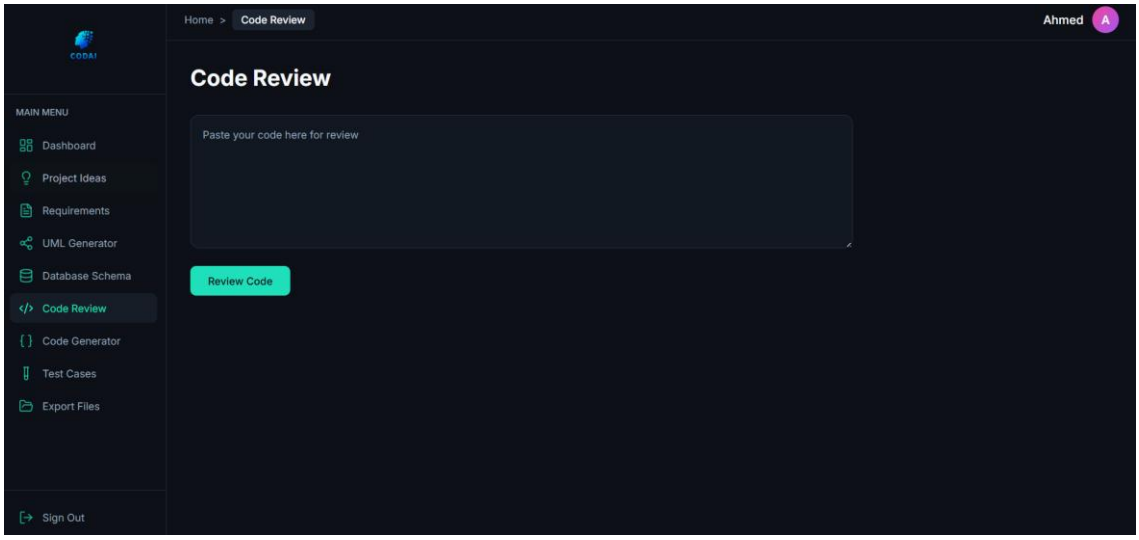


Figure 4.9: Code Review Interface

This module allows students to paste or upload code snippets. The AI reviews the code and provides feedback on potential issues, logic errors, and optimization opportunities. This feature supports learning through guided debugging and helps students improve their programming skills.

### Test Case Generation Screen

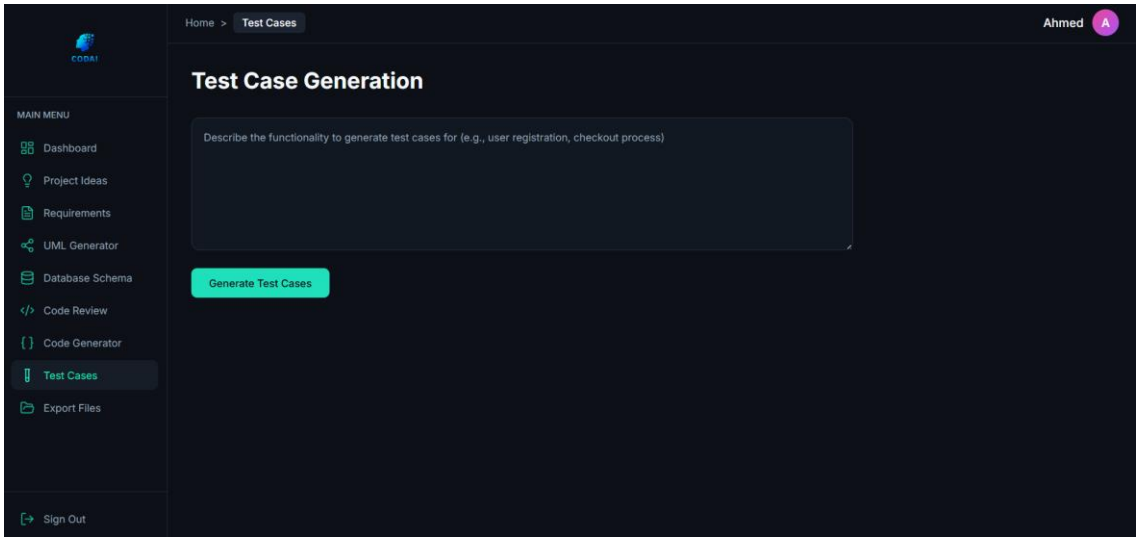


Figure 4.10: Test Case Generation Interface

Based on functional requirements or specific function descriptions, the system generates basic test cases. This helps students understand the relationship between requirements and testing and introduces them to structured test case development.

### Export and File Management Screen

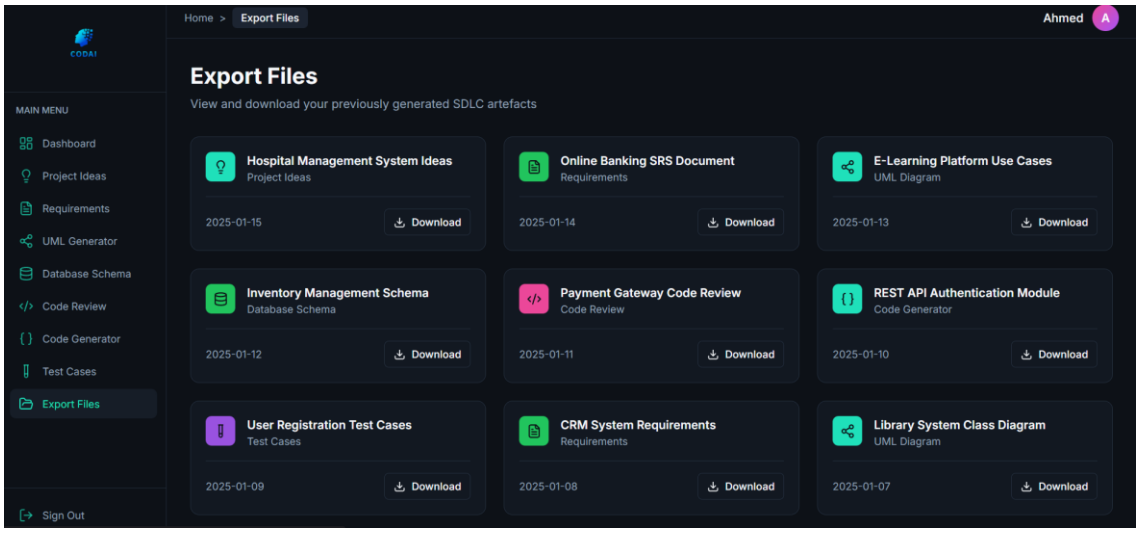


Figure 4.11: Export and File Management Interface

This screen allows students to download generated artefacts, including requirements, UML descriptions, database schema suggestions, test cases, and code review outputs. Files are stored using Firebase Cloud Storage and can be exported in formats such as PDF or text, supporting academic submissions and documentation work.

## 4.4 Database Design

### 4.4.1 Entity Relationship Diagram (ERD)

The Entity Relationship Diagram (ERD) shown above illustrates the database structure used for the AI-Powered SDLC Assistant. It models all the key data entities such as users, projects, requirements, UML diagrams, database tables, test cases, code artefacts, and generated reviews. The ERD ensures that each module within the system is supported with a consistent and scalable data model.

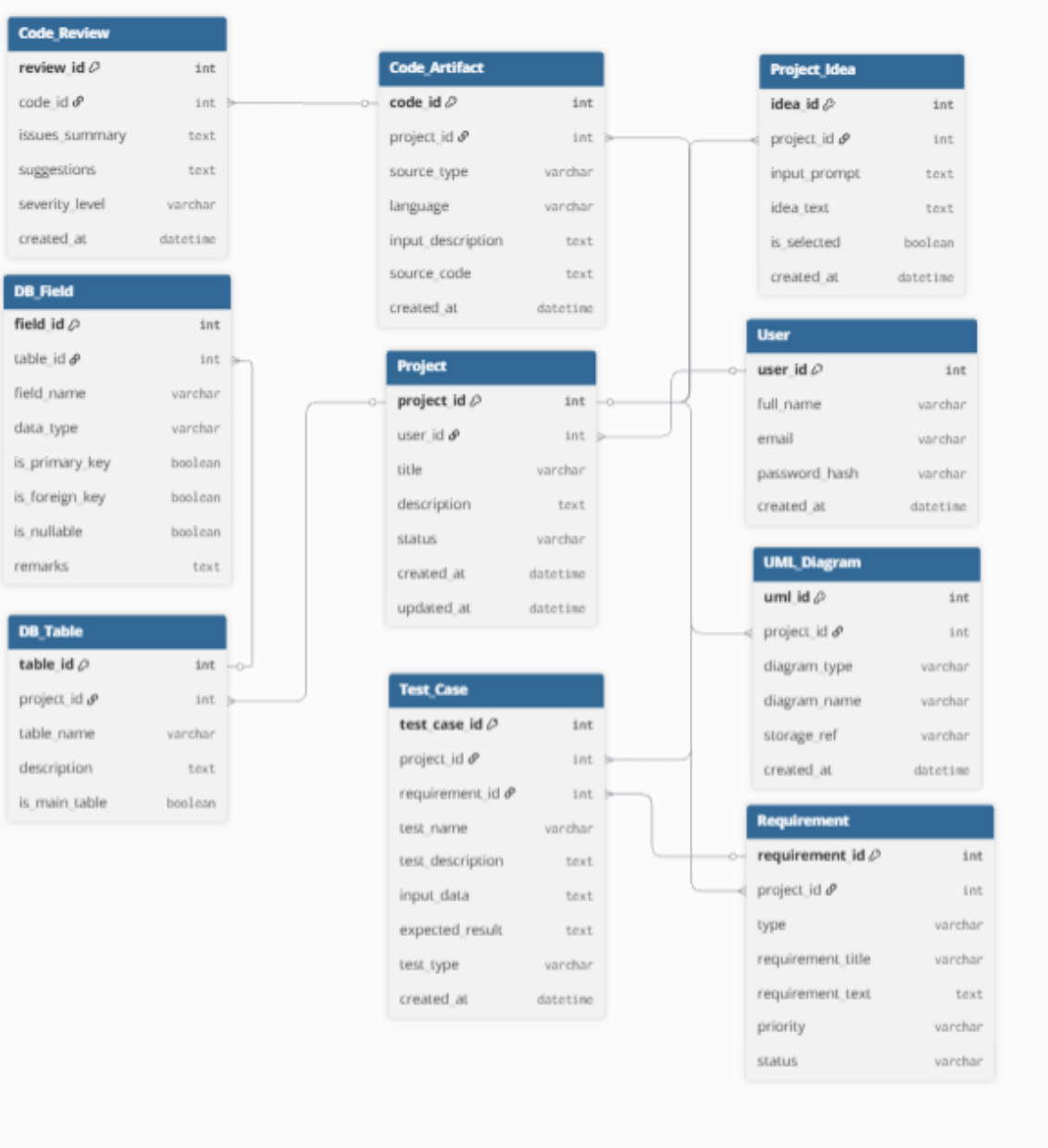


Figure 4.12: Entity Relationship Diagram

### 4.4.2 Data Dictionary

**Table: User**

Table 4.1: Data Dictionary for User Entity

| Field | Type | Key | Description |
|---|---|---|---|
| user_id | int | PK | Unique identifier for each user |
| full_name | varchar | - | User's full name |
| email | varchar | Unique | User login email |
| password_hash | varchar | - | Encrypted password |
| created_at | datetime | - | Account creation timestamp |

**Table: Project**

Table 4.2: Data Dictionary for Project Entity

| Field | Type | Key | Description |
|---|---|---|---|
| project_id | int | PK | Unique project identifier |
| user_id | int | FK | Links project to user |
| title | varchar | - | Project title |
| description | text | - | Short project description |
| status | varchar | - | Project progress status |
| created_at | datetime | - | Creation timestamp |
| updated_at | datetime | - | Last updated timestamp |

**Table: Project_Idea**

Table 4.3: Data Dictionary for Project_Idea Entity

| Field | Type | Key | Description |
|---|---|---|---|
| idea_id | int | PK | Unique idea identifier |
| project_id | int | FK | Belongs to project |
| input_prompt | text | - | User input for idea generation |
| idea_text | text | - | Generated idea |
| is_selected | boolean | - | Whether user selected this idea |
| created_at | datetime | - | Timestamp |

**Table: Requirement**

Table 4.4: Data Dictionary for Requirement Entity

| Field | Type | Key | Description |
|---|---|---|---|
| requirement_id | int | PK | Unique requirement ID |
| project_id | int | FK | Belongs to project |
| type | varchar | - | FR or NFR |
| requirement_title | varchar | - | Short title |
| requirement_text | text | - | Full requirement description |
| priority | varchar | - | Importance level |
| status | varchar | - | Requirement status |

**Table: UML_Diagram**

Table 4.5: Data Dictionary for UML_Diagram Entity

| Field | Type | Key | Description |
|---|---|---|---|
| uml_id | int | PK | Unique diagram ID |
| project_id | int | FK | Belongs to project |
| diagram_type | varchar | - | Use Case, Class, etc. |
| diagram_name | varchar | - | File name/title |
| storage_ref | varchar | - | Path/URL of diagram |
| created_at | datetime | - | Timestamp |

**Table: DB_Table**

Table 4.6: Data Dictionary for DB_Table Entity

| Field | Type | Key | Description |
|---|---|---|---|
| table_id | int | PK | Unique table ID |
| project_id | int | FK | Belongs to project |
| table_name | varchar | - | Name of the table |
| description | text | - | Table description |
| is_main_table | boolean | - | Whether it is a core table |

**Table: DB_Field**

Table 4.7: Data Dictionary for DB_Field Entity

| Field | Type | Key | Description |
|---|---|---|---|
| field_id | int | PK | Field identifier |
| table_id | int | FK | Links to DB_Table |

| | | | |
|---|---|---|---|
| field_name | varchar | - | Name of field |
| data_type | varchar | - | Data type |
| is_primary_key | boolean | - | Whether it is PK |
| is_foreign_key | boolean | - | Whether it is FK |
| is_nullable | boolean | - | Allows NULL |
| remarks | text | - | Additional info |

**Table: Code_Artifact**

Table 4.8: Data Dictionary for Code_Artifact Entity

| Field | Type | Key | Description |
|---|---|---|---|
| code_id | int | PK | Unique code ID |
| project_id | int | FK | Belongs to project |
| source_type | varchar | - | Generated or uploaded |
| language | varchar | - | Programming language |
| input_description | text | - | User's description |
| source_code | text | - | Code content |
| created_at | datetime | - | Timestamp |

**Table: Code_Review**

Table 4.9: Data Dictionary for Code_Review Entity

| Field | Type | Key | Description |
|---|---|---|---|
| review_id | int | PK | Review identifier |
| code_id | int | FK | Belongs to Code_Artifact |
| issues_summary | text | - | Identified issues |
| suggestions | text | - | AI improvement suggestions |
| severity_level | varchar | - | Severity rating |
| created_at | datetime | - | Timestamp |

**Table: Code_Review**

Table 4.10: Data Dictionary for Test_Case Entity

| Field | Type | Key | Description |
|---|---|---|---|
| test_case_id | int | PK | Unique test case ID |
| project_id | int | FK | Belongs to project |
| requirement_id | int | FK | Optional link to requirement |

| | | | |
|---|---|---|---|
| test_name | varchar | - | Name of test case |
| test_description | text | - | Description |
| input_data | text | - | Input values |
| expected_result | text | - | Expected output |
| test_type | varchar | - | Type (unit, functional) |
| created_at | datetime | - | Timestamp |

# CHAPTER 5

## 5.1 CONCLUSION

The development of the AI-Powered Software Engineering Study Assistant demonstrates the significant potential of artificial intelligence in improving learning effectiveness, productivity, and academic performance among software engineering students. Through surveys, interviews, and literature review, this project identified several recurring challenges faced by students, including difficulties in generating project ideas, writing structured requirements, modelling UML diagrams, designing database schemas, reviewing code, and producing meaningful test cases. Existing AI tools do not provide a fully integrated, SDLC-aware learning environment capable of addressing these needs holistically.

The proposed system addresses these gaps by centralizing key SDLC learning support features within a single AI-powered platform. With the integration of a Large Language Model (LLM), the system can generate project ideas, refine requirements, assist with UML modelling, recommend database structures, review code, and create automated test cases. Unlike general-purpose AI assistants, this system is tailored specifically for academic software engineering contexts, offering structured, context-aware guidance that helps students understand both the theory and practice of the SDLC.

The system design, which includes architecture, interface layouts, database schema, and requirement specifications, forms a strong foundation for the development phase in Final Year Project Phase 2 (FYP2). These components ensure that the system is scalable, secure, usable, and aligned with software engineering best practices. With a structured design approach, the next phase can proceed smoothly into implementation and testing.

In conclusion, the AI-Powered Software Engineering Study Assistant has the potential to significantly enhance the learning experience by providing personalized and adaptive guidance across multiple SDLC stages. It bridges the gap between theoretical knowledge and practical application, supporting students in developing stronger analytical, design, and problem-solving skills. This project also sets the groundwork for future AI-driven educational tools that can further improve the software engineering learning process.

## 5.2 Implementation Plan - FYP2

The implementation of the AI-Powered Software Engineering Study Assistant will be carried out during **Final Year Project Phase 2 (FYP2)**. The work in FYP2 will begin with setting up the complete development environment, including the frontend interface, backend server, Firebase authentication and database services, and integration with the AI API. This phase will ensure that all system components are properly configured and ready for development.

Following this, the core functional modules will be implemented incrementally. These modules include project idea generation, requirements analysis, UML diagram

generation, database schema recommendation, code generation, code review, and automated test case generation. Each module will be integrated with the AI engine to provide structured outputs and educational explanations for students.

In the final stage of FYP2, file export functionality will be implemented to allow users to download generated artefacts for academic use. Basic system testing and validation will then be conducted to ensure the system functions correctly, meets usability requirements, and aligns with the project objectives before final submission.

# REFERENCES


1. GitHub. (2023). *GitHub Copilot: Your AI pair programmer*. GitHub, Inc. Retrieved from https://github.com/features/copilot
2. OpenAI. (2024). *ChatGPT: Conversational AI model powered by GPT-4*. OpenAI. Retrieved from https://chat.openai.com/
3. Ask SIA. (2023). *Ask SIA (Student Intelligent Assistant)*. Retrieved from https://asksia.ai/
4. Visual Paradigm International Ltd. (2024). *Visual Paradigm – Professional UML and design tool*. Visual Paradigm International. Retrieved from https://www.visual-paradigm.com/
5. University of Kent. (2023). *BlueJ: A simple IDE for Java education*. University of Kent. Retrieved from https://www.bluej.org/
6. Firebase. (2024). Firebase Documentation. Google LLC. Retrieved from https://firebase.google.com/docs
7. Object Management Group. (2017). Unified Modeling Language (UML) Specification, Version 2.5.1. Retrieved from https://www.omg.org/spec/UML/2.5.1
8. Pressman, R. S., & Maxim, B. R. (2014). Software Engineering: A Practitioner's Approach (8th ed.). McGraw-Hill.
9. Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education.
10. Wiegers, K., & Beatty, J. (2013). Software Requirements (3rd ed.). Microsoft Press.
11. Myers, G. J., Sandler, C., & Badgett, T. (2011). The Art of Software Testing (3rd ed.). Wiley.

# APPENDIX A
# SAMPLE APPENDIX 1

For Appendices Heading use TITLE AT ROMAN PAGES style.

# APPENDIX B
# SAMPLE APPENDIX 2

For Appendices Heading use TITLE AT ROMAN PAGES style.